

Chapter-1

Possible Questions

1.1 Review of Python Essentials and Coding Conventions

1. What is PEP 8 and why is it important in Python development?
2. Explain the difference between Python 2 and Python 3.
3. How does Python manage memory for variables?
4. Write a Python function to check if a string is a palindrome.
5. What are Python keywords? List any 5 of them.
6. Explain the difference between mutable and immutable data types in Python.
7. What is the difference between `is` and `==` operators in Python?
8. Describe Python's naming conventions for variables, functions, and classes.
9. How do you write a multi-line comment in Python?
10. Explain the difference between shallow copy and deep copy in Python.

1.2 Advanced Data Structures: Collections, Iterators, Generators, and Decorators

1. What is the difference between a list, tuple, set, and dictionary in Python?
2. Explain the difference between `defaultdict` and `Counter` from the `collections` module.
3. What are Python iterators? How do you create one?
4. Write a generator function to yield Fibonacci numbers up to `n`.
5. Explain how a decorator works in Python with an example.
6. What is the difference between a generator and a list comprehension?
7. How does the `chain` function from `itertools` work?
8. Explain the difference between `map()`, `filter()`, and `reduce()`.
9. How can you reverse iterate over a list without creating a copy?
10. Write a decorator to measure the execution time of a function.

1.3 Functions and Lambda Expressions

1. What is the difference between a function and a lambda expression in Python?
2. Write a lambda function to square a number.
3. Explain the difference between positional, keyword, and default arguments in Python functions.
4. What is variable scope and the difference between local and global variables?
5. How do you return multiple values from a Python function?
6. Explain the use of `*args` and `**kwargs` with examples.
7. What is a higher-order function in Python? Give an example.
8. How does a recursive function work? Write a recursive function to compute factorial.
9. What is the difference between `return` and `yield` in Python functions?
10. How can you use a lambda function with `map()` and `filter()`?

1.4 Object-Oriented Programming for Data Science Applications

1. What are the four main principles of OOP? Explain each with Python examples.
2. How do you define a class and create an object in Python?
3. What is the difference between class variables and instance variables?
4. Explain method overriding and method overloading in Python.
5. What is the purpose of the `__init__` method?
6. Explain the difference between public, protected, and private attributes in Python.
7. How do you implement inheritance in Python? Give an example.
8. What is polymorphism? Demonstrate it with Python code.
9. How can OOP be used to structure a data analysis pipeline?
10. What are magic methods (`__str__`, `__repr__`, `__len__`)? Give examples.

1.5 Exception Handling, Debugging, and Logging

1. What is the difference between `SyntaxError` and `Exception` in Python?
2. How do you handle exceptions using `try`, `except`, `else`, and `finally`?
3. Explain the difference between `raise` and `assert` in Python.
4. Write Python code to handle division by zero exception.
5. How do you create a custom exception in Python?
6. What are some common debugging techniques in Python?
7. How does the logging module work? Give an example of logging info and error messages.
8. Explain the difference between `print()` and logging for debugging purposes.
9. What is the use of `traceback` in Python?
10. How can you configure logging to write logs to a file instead of the console?

1.6 Working with Modules and Packages

1. What is the difference between a module and a package in Python?
2. How do you import a module in Python? Give multiple ways.
3. Explain the purpose of `__init__.py` in a Python package.
4. How do you list all functions and variables in a module?
5. How can you reload a module in Python after changes?
6. Explain the difference between absolute and relative imports.
7. How do you install external packages using `pip`?
8. Write Python code to import only specific functions from a module.
9. What is the difference between standard library modules and third-party modules?
10. How can you create your own Python package and make it reusable?